

Scaffolding Computational Thinking for Non-IT Graduate Researchers: A Rapid Pedagogical Intervention.

Alexandre Cabral¹ , Cayo Andrade¹ , Danilo Silva¹ , Felipe Lima¹  and Malu Silva¹ 

¹*Informatics Center, Federal University of Pernambuco, Recife, Brazil*
{asc5, crma, dfss, fal6, mxs}@cin.ufpe.br

Keywords:

Abstract: The increasing digitalization of interdisciplinary research demands computational skills from non-Information Technology (IT) graduate students. However, traditional computing education often emphasizes programming syntax, leading to cognitive overload and severe emotional distress among these learners. This experience report details the design and evaluation of a 4-hour active learning intervention aimed at developing foundational computational thinking for non-CS majors. By intentionally decoupling algorithmic problem-solving from syntax mechanics, the course utilized unplugged guided inquiry for everyday problems (e.g., the "cuscuz" algorithm) and visual block-based programming via Google Blockly within a collaborative pair-programming environment. A mixed-methods approach, including formative and summative assessments, was used to evaluate student self-efficacy and perceptions. The results demonstrated maximum student satisfaction, with all participants successfully translating logical reasoning into functional visual code and performing autonomous debugging. We conclude that prioritizing accessible language, cognitive scaffolds, and socio-emotional peer support empowers interdisciplinary researchers to rapidly acquire robust mental models of program structure and dynamic flow, thereby effectively mitigating the frustrations typically associated with novice programming.

¹ <https://orcid.org/0000-0000-0000-0000>

² <https://orcid.org/0000-0000-0000-0000>

³ <https://orcid.org/0000-0000-0000-0000>

⁴ <https://orcid.org/0000-0002-0000-9502>

⁵ <https://orcid.org/0000-0000-0000-0000>

1 INTRODUCTION

1.2 Context & Motivation

The increasing digitalization of research across diverse academic disciplines—ranging from biosciences and psychology to business and education—has made computational thinking and programming foundational competencies for modern researchers (Barman, Beckman, & Chebaro, 2022; Mike & Hazzan, 2022; Xu & Babaian, 2021). However, non-Information Technology (IT) graduate students, who are often "latecomers" to computing (Lehman et al., 2020), face significant technical and socio-emotional barriers when attempting to acquire these skills. Traditional computing courses tend to emphasize syntax and low-level programming mechanics, which frequently results in cognitive overload and ineffective trial-and-error behaviors. For these non-CS majors, the experience of learning to program and debug code can evoke severe emotional distress and isolation, provocatively described by students as inducing "a river of tears" (Tarter et al., 2025). Consequently, without a clear understanding of the static structure and dynamic flow of programs, many students fail to see the relevance of computing to their domains and disengage (Hur et al., 2025). Addressing this gap requires shifting the pedagogical focus from syntax memorization to the structured development of computational thinking tailored to the specific needs and time constraints of interdisciplinary graduate students.

1.3 Theoretical Foundation & Strategy

To tackle these challenges, we designed and implemented a 4-hour introductory mini-course titled "Introdução ao Pensamento Computacional", targeted at graduate students without a computing background. Our pedagogical strategy is grounded in active learning, cognitive scaffolding, and collaborative practices. Instead of immediately exposing students to text-based coding, we utilized Google Blockly, a visual block-based programming environment that eliminates syntax errors and reduces cognitive load, allowing learners to focus entirely on algorithmic logic and problem-solving (Zhang & Rutherford, 2024). The intervention was structured around the four pillars of computational thinking (Decomposition, Pattern Recognition, Abstraction, and Algorithmic Thinking). To bridge

the socio-emotional gaps, we integrated Guided Inquiry methodologies (Bettin et al., 2022)—prompting students to translate everyday problems (e.g., making couscous or calculating grades) into structured pseudo-code—and Collaborative Learning strategies, specifically Pair Programming (Driver & Navigator roles), to provide immediate peer support, foster technical communication, and reduce the fear of making mistakes (Clarke et al., 2020).

1.4 Objectives & Scope

The primary objective of this experience report is to explore how this short-duration, highly scaffolded pedagogical intervention supports the initial computing education of non-IT graduate students. Specifically, we aim to evaluate the effectiveness of combining unplugged algorithmic modeling with visual block-based programming (Blockly) in helping students translate logical reasoning into functional code. Furthermore, the report investigates the students' perceptions of the active learning strategies applied (such as pair programming and guided inquiry) and identifies the benefits and limitations of this instructional design in mitigating technical frustration and fostering self-efficacy.

1.5 Document Structure & Contributions

The main contributions of this report include the detailed design of a highly contextualized, rapid-intervention instructional model tailored for graduate researchers, alongside an empirical analysis of their qualitative and quantitative reflections post-intervention. The remainder of this document is organized as follows: Section 2 discusses the background and related work regarding non-IT computing education and block-based scaffolding. Section 3 details the specific course context and participant demographics. Section 4 outlines the core pedagogical characteristics and architectural design of the mini-course. Section 5 describes the methodology for data collection and analysis. Section 6 presents the results of the student evaluations. Section 7 discusses the findings and provides actionable recommendations. Finally, Section 8 summarizes the key takeaways and outlines future work.

2 BACKGROUND AND RELATED WORK

2.1 Educational Paradigm

The pedagogical design of our intervention is rooted in active learning, cognitive scaffolding, and collaborative strategies. Active learning shifts the focus from passive knowledge reception to dynamic student engagement. In computing education, the integration of Learning and Engagement Strategies (LESSs)—such as collaborative learning and problem-based learning—has been shown to maximize student understanding and knowledge acquisition through meaningful social interaction (Clarke, Allala, Sotomayor, & Ross, 2020). We operationalized collaborative learning primarily through Pair Programming, a practice where two students work together at one workstation as "driver" and "navigator". This approach not only mimics real-world software engineering practices but also mitigates the isolation and anxiety novices often feel when encountering syntax errors.

To further reduce the initial cognitive load, our paradigm relies heavily on cognitive scaffolding through visual block-based programming. Block-based environments, such as Google Blockly, provide a low-floor entry point that eliminates syntax errors and reduces the degrees of freedom, allowing learners to focus purely on problem-solving, algorithmic logic, and structural comprehension (Zhang & Rutherford, 2024). Complementing this technological scaffold, we utilized Guided Inquiry methodologies for unplugged activities. By prompting students to outline algorithms for everyday problems before writing any code, Guided Inquiry helps learners conceptualize programming in a language-independent way, alleviating confusion regarding static program structure and dynamic program flow (Bettin, Jarvie-Eggart, Steelman, & Wallace, 2022).

2.2 State-of-the-Art in Computing Education

The state-of-the-art in computing education has increasingly recognized the unique challenges faced by non-Information Technology (IT) majors. Non-CS graduate students are often "latecomers" to computing who must balance the acquisition of complex technical skills with the severe time constraints of their primary research domains (Hur,

Kang, Park, & Cunningham, 2025). The emotional toll of this learning curve is significant; studies show that social science graduate students frequently experience severe distress—sometimes describing the process of learning to code for research as inducing "a river of tears"—due to a lack of targeted guidance, camaraderie, and transferability of the computing skills to their specific contexts (Tarter, Zhou, Yarosh, & DeLiema, 2025).

To address these barriers, recent initiatives have explored highly contextualized interdisciplinary CS1 courses. For instance, Mike and Hazzan (2022) developed a tailored introductory course for graduate psychology students that replaced generic datasets with real-world psychological data. Grounded in the self-determination theory, their approach successfully fostered student autonomy, competence, and relatedness. Similarly, Barman, Beckman, and Chebaro (2022) implemented a programming and data science course specifically for postdoctoral researchers in the biosciences. Their findings underline that successful continuing education for advanced researchers must leverage experiential learning tied directly to the scholars' own datasets. In the business domain, Xu and Babaian (2021) demonstrated that the key challenge in teaching technologies like Artificial Intelligence to non-technical audiences is finding the right balance—providing a high-level conceptual perspective supported by just enough technical detail, avoiding the overly granular focus typical of traditional computer science curricula.

2.3 Research Gap and Novelty

Despite the growing body of literature on interdisciplinary computing education, a significant gap remains. Most existing models either propose semester-long CS1 adaptations that conflict with the severe time constraints and immediate research needs of graduate students (Barman, Beckman, & Chebaro, 2022), or they manifest as isolated, tool-specific workshops (e.g., "How to use SPSS" or "Intro to Python") that bypass the foundational development of computational thinking. Consequently, students often resort to inefficient trial-and-error behaviors because they learn the syntax without grasping the underlying algorithmic logic (Bettin, Jarvie-Eggart, Steelman, & Wallace, 2022).

Our work explicitly bridges this gap by proposing a rapid, 4-hour pedagogical intervention that intentionally decouples logical problem-solving from syntax execution. The novelty of this

experience lies in sequentially integrating unplugged algorithm translation (guided inquiry) with visual block-based programming (Blockly) within a highly collaborative (pair programming) environment for graduate students. Instead of jumping straight into text-based coding, this framework forces interdisciplinary researchers to first consolidate their mental models of decomposition, abstraction, and dynamic flow. By addressing the socio-emotional barriers of coding while simultaneously scaffolding the cognitive transition from everyday logic to functional algorithms, this instructional model provides a distinct, efficient pathway to build research computing capabilities without the initial burden of syntax memorization.

3 COURSE CONTEXT

3.1 Institutional Context

The pedagogical intervention was conducted as a 4-hour mini-course titled "Introdução ao Pensamento Computacional" during the 2026.1 academic semester at the Informatics Center (Centro de Informática) of the Federal University of Pernambuco (UFPE). The target audience consisted exclusively of graduate students without prior computing backgrounds. From an initial pool of 17 registered applicants, 7 students participated in the session. The cohort was highly interdisciplinary, comprising Master's (n=5) and Doctoral (n=2) students from diverse fields, including Information Science, Administration, Tourism, Education, Geology, Architecture and Urbanism, and Letters. The demographic profile highlighted a mature audience, with the majority of participants aged between 31 and over 41 years old, who balance their academic research with professional roles in teaching, public management, and planning.

3.2 Curriculum Integration

Designed as an extracurricular, rapid-intervention module (linked to the advanced topics discipline IN1169), the mini-course deliberately diverged from traditional semester-long programming models. Acknowledging the severe time constraints typical of graduate researchers, the instructional mechanisms shifted from passive, semester-long lectures to a single, highly concentrated active learning session. Traditional summative assessments, such as written exams, were entirely

replaced by continuous formative evaluation. Student progress was assessed in real-time through their engagement in practical, hands-on challenges (both unplugged and digitally using visual blocks) and through immediate peer and instructor feedback.

3.3 Conceptual Architecture

The baseline subject matter was structured around a progressive, four-step learning trajectory designed to mitigate cognitive overload. First, students were introduced to the four core pillars of computational thinking: Decomposition, Pattern Recognition, Abstraction, and Algorithmic Thinking. Second, students engaged in Guided Inquiry through unplugged activities, translating everyday real-world tasks—such as the algorithm to prepare a regional dish ("cuscuz") or calculating academic grades—into logical pseudo-code. Third, learners transitioned to Google Blockly, utilizing a block-based visual environment to construct functional code for mathematical challenges, thereby eliminating the burden of syntax. Finally, the intervention concluded with an analytical phase where students tested and debugged their logic, accompanied by a brief introduction to text-based programming in Python to bridge the gap toward real-world research applications.

3.4 Identified Challenges

The necessity for this tailored context stemmed from historical and empirical evidence regarding non-IT learners. Traditional computing courses frequently induce cognitive overload and emotional distress—often resulting in ineffective "trial-and-error" behaviors—due to a premature focus on programming language syntax (Bettin, Jarvie-Eggart, Steelman, & Wallace, 2022; Tarter, Zhou, Yarosh, & DeLiema, 2025). Furthermore, a preliminary analysis conducted by our pedagogical team using the ADDIE model revealed that while these graduate students possess strong theoretical domain knowledge and statistical foundations, they severely lack the structured mental models required to comprehend static code structures and dynamic execution flows. This gap traditionally paralyzes their independent research capabilities when faced with computational bugs. Consequently, this course context was explicitly designed to dismantle these socio-emotional and technical barriers by leveraging highly scaffolded, syntax-free environments before

introducing formal coding (Zhang & Rutherford, 2024).

4 COURSE DESIGN

4.1 Architectural Overview

The pedagogical intervention was structured as a single, intensive 4-hour session, systematically divided into progressive learning blocks to prevent cognitive overload. Following the ADDIE model's structured approach to instructional design (Peterson, 2003), the architecture deliberately transitioned students from language-independent logic to functional code. The session was segmented into four main trajectories: (1) Understanding the four pillars of computational thinking; (2) Translating real-world problems through unplugged activities; (3) Visual block-based programming using Google Blockly; and (4) Understanding, analyzing, and testing text-based syntax via Python. This gradual scaffolding ensured that students first consolidated their mental models of algorithms before encountering the strict rigidity of programming syntax (Zhang & Rutherford, 2024).

4.2 Core Pedagogical Characteristics

To effectively address the technical and socio-emotional barriers faced by non-IT graduate students, the intervention relied on three fundamental pedagogical pillars:

- **Guided Inquiry and Unplugged Activities:**
Before interacting with any software, students engaged in Guided Inquiry (Bettin, Jarvie-Eggart, Steelman, & Wallace, 2022). They mapped the inputs, processes, and outputs of everyday tasks, such as the algorithm to prepare a regional dish ("cuscuz") and the calculation of academic grades. This allowed them to practice decomposition and abstraction in a familiar context without technical intimidation.
- **Cognitive Scaffolding via Visual Blocks:**
The transition to digital programming utilized Google Blockly, drawing inspiration from the Parsons Method. By providing predefined visual logic blocks, the environment eliminated syntax errors and allowed students to focus purely on the sequence, structure, and

dynamic flow of control structures like conditionals and loops.

- **Collaborative Learning (Pair Programming):** To mitigate the isolation and emotional distress typically associated with novice programming (Tarter, Zhou, Yarosh, & DeLiema, 2025), students worked collaboratively in pairs following the "Driver and Navigator" roles. This active learning strategy fostered technical communication, provided immediate peer-to-peer support, and built the confidence required to debug errors collectively rather than resorting to trial-and-error (Clarke, Allala, Sotomayor, & Ross, 2020).

4.3 Timeline and Practical Dynamics

The 4-hour timeline was meticulously orchestrated to maintain high engagement. The first hour was dedicated to an unplugged "Do It Now!" (Faça Agora!) challenge where students mapped the "Cuscuz Algorithm" using pen and paper, followed by a debriefing on the four pillars of computational thinking (Decomposition, Pattern Recognition, Abstraction, and Algorithmic Thinking). The second hour shifted to a more analytical translation task, where students wrote pseudo-code to calculate an average academic grade and assign an educational concept.

During the third hour, students transitioned to the digital environment using Google Blockly for hands-on pair programming challenges. These challenges included calculating the steps of a staircase and building an interactive multiplication table, forcing them to apply loops and conditional logic in a syntax-free playground. The final hour served as an analytical synthesis, where the visual logic constructed in Blockly was directly mapped to fundamental Python syntax (variables, arithmetic operators, conditionals, and loops). The session culminated in a final debugging reflection and an "arremate" (wrap-up) exercise in Python, with formative feedback continuously provided by the instructional team throughout all stages to ensure objectives were being met (Peterson, 2003).

5 METHODOLOGY

5.1 Data Collection

To rigorously evaluate the efficacy of the instructional design, we utilized a mixed-methods approach to gather continuous formative and summative feedback from the participants. The primary data sources were four targeted questionnaires administered via Google Forms during and after the session. Three formative surveys were deployed immediately following the core practical exercises (the unplugged "Cuscuz" algorithm, the unplugged "Average Grade" calculation, and the digital Google Blockly challenges). These surveys utilized 5-point Likert scales to capture real-time perceptions of cognitive load, instruction clarity, and algorithmic confidence. A comprehensive summative evaluation survey was administered at the end of the 4-hour intervention. This final instrument collected quantitative data regarding the course format, pedagogical team, and infrastructure, alongside qualitative data through open-ended questions where students detailed the perceived strengths and areas for improvement. Furthermore, student-generated pseudo-codes submitted on Padlet digital boards were collected as observational artifacts to document their logical translation process.

5.2 Ethical Considerations & Anonymization

The data collection process adhered strictly to ethical guidelines for educational research to protect participant privacy and ensure data integrity. Prior to completing the final evaluation survey, all participants were presented with a formal Data Sharing Consent agreement (Consentimento de Compartilhamento de Dados). The consent explicitly stated that participation in the research was entirely voluntary, that responses would be used exclusively for scientific purposes, and that students could opt out at any moment without justification or penalty. Although emails and names were initially recorded for administrative purposes (such as tracking attendance and issuing course certificates), all personally identifiable information was completely anonymized and stripped from the datasets prior to the analysis phase. The aggregated data were stored in a secure environment to guarantee confidentiality.

5.3 Data Analysis Strategy

The analysis of the collected data followed a systematic, mixed-methods strategy to evaluate both

the implementation and evaluation phases of the ADDIE model (Peterson, 2003). Quantitative responses from the Likert-scale questions across all four surveys were analyzed using descriptive statistics (frequencies and percentages) to objectively measure the students' self-efficacy, structural comprehension, and satisfaction at each pedagogical stage.

For the qualitative data—comprising the open-ended survey comments and the algorithmic logic mapped on the Padlet boards—we employed a reflexive, data-driven thematic analysis approach (Braun & Clarke, 2006). The pedagogical team first familiarized themselves with the raw textual responses. Next, we conducted inductive coding to identify recurring patterns in the students' reflections, capturing specific pain points and highlights (e.g., identifying codes for "accessible didactics" and "infrastructure bottlenecks"). Finally, these codes were synthesized into overarching evaluative themes to critically assess the impact of the cognitive scaffolds and the collaborative learning environment on the students' ability to learn computational thinking.

6 RESULTS

6.1 Empirical Overview

The 4-hour pedagogical intervention yielded overwhelmingly positive quantitative outcomes regarding student perception and self-efficacy. Formative assessments conducted immediately after the Google Blockly practical block revealed that 100% of the respondents either agreed (83.3%) or strongly agreed (16.7%) that they successfully translated their logical reasoning into the visual environment, comprehending how the arrangement of blocks defines the program's execution order. Furthermore, regarding debugging—a task traditionally paralyzing for novices—100% of the students (66.7% agreed, 33.3% strongly agreed) reported being capable of autonomously re-reading their logic, identifying errors, and correcting the blocks. In the comprehensive summative evaluation, maximum satisfaction was recorded: all participants strongly agreed (5 on a 5-point Likert scale) that the division of the mini-course into practical blocks and theoretical moments facilitated their learning experience and helped them understand the definitions of computational thinking.

6.2 Theme Deep Dive

To complement the quantitative data, a thematic analysis of the students' open-ended reflections at the end of the course identified three core strengths: Accessible Didactics, Practical Application, and Course Organization.

- **Accessible Didactics and Language:** Students frequently praised the instructional approach, noting the "accessible language and lightness in the exposition of concepts and examples." This directly counteracts the severe emotional distress and intimidation often experienced by non-CS graduate students when facing traditional computing jargon (Tarter, Zhou, Yarosh, & DeLiema, 2025).
- **Practical Application (Scaffolding):** The use of visual programming and continuous peer/tutor support was highlighted as a major catalyst for learning. One student summarized the pedagogical dynamic as an "excellent class: dialogic, participatory, and very practical."
- **Relevance to Research:** The contextualization of algorithms demonstrated clear value to the researchers. A participant noted: "The mini-course was very fruitful and contributed significantly to understanding the foundations of computational thinking and its applications in the educational context. The adopted methodology favored interaction among participants and stimulated reflection on strategies to develop problem-solving skills."

6.3 Progression of Algorithmic Confidence

The formative data captured a clear evolution in student confidence as they transitioned from unplugged activities to digital scaffolding. During the initial unplugged "Cuscuz" algorithm challenge, while students found it relatively easy to break down the task, their confidence in the algorithm's absolute success was moderate, with several students expressing doubts about whether they had mapped every essential detail. However, after engaging with the Parsons Method and transitioning to the syntax-free environment of Google Blockly, self-efficacy spiked significantly. When asked about

utilizing conditional blocks (If/Else) to evaluate data and make decisions, 83.3% of students rated their understanding at the highest levels (4 or 5 out of 5), demonstrating that the cognitive scaffold effectively bridged the gap between abstract logic and programmatic execution (Zhang & Rutherford, 2024).

6.4 Identified Limitations and Bottlenecks

Despite the highly positive reception—with one student calling the course "a pleasant surprise"—the participants transparently identified specific structural limitations that offer actionable insights for future iterations. The primary bottleneck identified was related to physical infrastructure. Students noted that the intervention took place in a standard classroom with a single multimedia projector ("datashow"), suggesting that a dedicated computer lab would have provided a more immersive and smoother hands-on experience.

Additionally, addressing the time constraints inherent to graduate researchers, multiple students requested an expanded workload for the course. They suggested that allocating more time specifically for practical activities and the resolution of coding challenges would allow for a deeper exploration of the content. Finally, recognizing the value of the intervention, participants suggested that the material should be recorded and made available online (e.g., via YouTube) to democratize access to this "fantastic didactics" for a broader audience of non-computing professionals, echoing the widespread demand for accessible interdisciplinary computing education (Barman, Beckman, & Chebaro, 2022).

7 DISCUSSION

7.1 Alignment with Educational Literature

The qualitative and quantitative results of this intervention strongly align with recent literature on computing education for non-majors. Historically, graduate researchers from other fields report that the programming learning curve can be emotionally exhausting, describing the experience as a "river of tears" due to isolation and frustration with syntax errors (Tarter, Zhou, Yarosh, & DeLiema, 2025).

Furthermore, literature demonstrates that the initial interest of these students often dissipates when courses fail to demonstrate personal relevance or provide unwelcoming learning environments (Hur, Kang, Park, & Cunningham, 2025).

Our intervention countered this attrition trend. The qualitative reports from the students (e.g., "accessible language and lightness in the exposition of concepts") indicate that the adoption of active learning and engagement strategies (Clarke, Allala, Sotomayor, & Ross, 2020) effectively reduced technological anxiety. The unanimous satisfaction with the course dynamics confirms the findings of Barman, Beckman, and Chebaro (2022), who argue that interdisciplinary computing education for researchers is only successful when the instruction is highly contextualized and shifts away from traditional methodologies focused purely on language mechanics.

7.2 Mechanism Deconstruction

The success of the intervention in developing the students' algorithmic self-efficacy can be attributed to the structural deconstruction of the teaching process, which intentionally separated computational logic from code syntax. Three central mechanisms catalyzed these higher-order cognitive skills:

- **Everyday Abstraction (Guided Inquiry):** By starting the course with the unplugged "Cuscuz" algorithm, we forced students to practice problem decomposition using familiar mental models. This corroborated the findings of Bettin, Jarvie-Eggart, Steelman, and Wallace (2022), proving that Guided Inquiry helps students conceptualize the static and dynamic structure of a program without the cognitive block of a computer screen.
- **The Technological Scaffold (Parsons Method and Blockly):** The transition from paper to Google Blockly was fundamental. Students reported extreme confidence (100% positive agreement) when testing loops and conditionals in practice. The use of predefined visual blocks (inspired by the Parsons Method) acted as a perfect cognitive scaffold, confirming that the elimination of typing and syntax errors allows the novice's brain to focus exclusively on problem-solving (Zhang & Rutherford, 2024).

- **Socio-emotional Support (Pair Programming):** Qualitative impacts revealed that the "effective mentoring" and "dialogic class" were high points. The requirement to work in pairs mitigated isolation and promoted camaraderie—an essential value for social science and humanities students (Tarter, Zhou, Yarosh, & DeLiema, 2025). Students were able to collaboratively discuss and debug errors, transforming code breakage from an individual failure experience into a collaborative resolution challenge.

7.3 Actionable Recommendations

Based on the reported bottlenecks and observed successes, we provide the following practical recommendations for educators seeking to integrate computational thinking into interdisciplinary graduate curricula:

- **Delay Syntax:** Never begin teaching programming to non-majors with text-based languages (such as Python or R). Use a structured bridge (Unplugged Activities → Visual Block-Based Programming → Text-Based Programming) to build algorithmic confidence first.
- **Contextualize Intensely:** Using algorithms that solve real-world problems (such as calculating averages or processing research data) translates the tool's value to the student's domain, which is vital for maintaining long-term engagement and interest (Hur, Kang, Park, & Cunningham, 2025).
- **Adapt Infrastructure and Time:** The main reported limitation was infrastructure (use of a single multimedia projector and limited practice time). Institutions must ensure that rapid interventions take place in dedicated computer labs and consider recording the instructional modules, allowing students to revisit concepts at their own pace while executing their independent research.

8 COPYRIGHT FORM

For the mutual benefit and protection of Authors and Publishers, it is necessary that Authors provide formal written Consent to Publish and Transfer of

Copyright before publication of the Book. The signed Consent ensures that the publisher has the Author's authorization to publish the Contribution.

The copyright form is located on the authors' reserved area.

The form should be completed and signed by one author on behalf of all the other authors.

9 CONCLUSIONS

9.1 Executive Summary

The increasing demand for computational skills in interdisciplinary research requires a paradigm shift in how computing is taught to non-Information Technology (IT) graduate students. This experience report detailed the design, implementation, and evaluation of a 4-hour active learning intervention aimed at developing foundational computational thinking for non-CS majors. By replacing traditional syntax-heavy lectures with a progressive, scaffolded approach—utilizing unplugged guided inquiry for everyday problems (e.g., the "cuscuz" algorithm) and visual block-based programming (Google Blockly) in a collaborative pair-programming environment—the intervention effectively mitigated the severe emotional distress and cognitive overload typically associated with novice programming (Tarter, Zhou, Yarosh, & DeLiema, 2025). Empirical findings from formative and summative assessments demonstrated maximum student satisfaction, with all participants successfully translating logical reasoning into functional visual code and demonstrating autonomous debugging capabilities.

9.2 Key Insight

The distinct value proposition of this pedagogical framework lies in its intentional decoupling of algorithmic problem-solving from programming syntax. Traditional computing courses often conflate these two cognitive processes, leading to inefficient trial-and-error behaviors and high attrition rates among "latecomers" to computing (Hur, Kang, Park, & Cunningham, 2025). By introducing cognitive scaffolds such as the Parsons Method and visual blocks, this model bridges the critical gap between mechanical tool execution and deep conceptual judgment (Zhang & Rutherford, 2024). It proves that interdisciplinary researchers can rapidly acquire robust mental models of static program structure and dynamic flow when the learning environment

prioritizes accessible language, practical contextualization, and socio-emotional peer support over immediate text-based coding.

9.3 Limitations and Future Work

Despite the highly positive outcomes, this study presents certain limitations. The intervention was conducted within a single institutional context with a relatively small cohort of graduate students, which restricts the broad generalizability of the empirical findings. Furthermore, the reliance on self-reported reflections and the severe time constraints (a single 4-hour session) limited the depth of exploration into text-based programming (Python). To overcome these barriers, it is recommended that future work focuses on expanding the course workload to allow for a deeper practical exploration of coding and on replicating the intervention in dedicated computer laboratories, thereby resolving the identified infrastructure bottlenecks. Additionally, it is suggested to produce and distribute similar instructional modules (e.g., via video platforms) to democratize access for a broader audience (Barman, Beckman, & Chebaro, 2022) and to incorporate extended quantitative metrics to track the long-term impact of this computational thinking foundation on the students' independent research trajectories throughout their academic training.

ACKNOWLEDGEMENTS

If any, should be placed before the references section without numbering.

REFERENCES

- Barman, A., Beckman, L. S., Chebaro, Y. (2022). Interdisciplinary computing education: An introductory programming and data science course for postdoctoral researchers in the biosciences. In FIE, 2022 IEEE Frontiers in Education Conference. IEEE.
- Bettin, B., Jarvie-Eggart, M., Steelman, K. S., Wallace, C. (2022). Preparing first-year engineering students to think about code: A guided inquiry approach. *IEEE Transactions on Education*, 65(3), 310-319. IEEE.
- Braun, V., Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77-101. Routledge.
- Clarke, P. J., Allala, S. C., Sotomayor, J. P., Ross, M. S. (2020). Experiences of integrating learning and engagement strategies (LESS) into software

- engineering courses. In ASEE 2020, ASEE Virtual Annual Conference. ASEE.
- Hur, J., Kang, M. M., Park, J., Cunningham, K. (2025). Applying the model of interest development to understand why non-CS majors decide to persist in or leave computing. In Koli Calling '25, 25th Koli Calling International Conference on Computing Education Research. ACM.
- Lehman, K. J., Wofford, A. M., Sendowski, M., Newhouse, K. N. S., Sax, L. J. (2020). Better late than never: Exploring students' pathways to computing in later stages of college. In SIGCSE '20, 51st ACM Technical Symposium on Computer Science Education. ACM.
- Mike, K., Hazzan, O. (2022). Interdisciplinary CS1 course for non-majors: The case of graduate psychology students. In EDUCON, 2022 IEEE Global Engineering Education Conference. IEEE.
- Peterson, C. (2003). Bringing ADDIE to life: Instructional design at its best. *Journal of Educational Multimedia and Hypermedia*, 12(3), 227-241. AACE.
- Tarter, A., Zhou, M., Yarosh, S., DeLiema, D. (2025). "A river of tears is a valid answer?": Social science graduate students' experiences learning programming for research. In Koli Calling '25, 25th Koli Calling International Conference on Computing Education Research. ACM.
- Xu, J. J., Babaian, T. (2021). Artificial intelligence in business curriculum: The pedagogy and learning outcomes. *The International Journal of Management Education*, 19(3), 100550. Elsevier.
- Zhang, Y., Rutherford, T. (2024). Scaffolding expertise: Evaluating scaffolds for block-based coding among experts and novices. In ISAIE 2024, 2024 International Symposium on Artificial Intelligence for Education. ACM.

APPENDIX

If any, the appendix should appear directly after the references without numbering, and not on a new page.